Monotonic Architectures for Lost-Sales Inventory Control

Section 1 - project objective and current compact result

Project description: replace loss-function penalties for monotonicity with architecturally enforced constraints, then test whether the resulting policy is better structured, faster to train, and more interpretable than penalty-based or unconstrained networks.

- Structure criterion: MonoKAN raw and deployed monotonicity violations are 0.000 and 0.000; this passes the architecture-enforced monotonicity check.
- Performance criterion: MonoKAN reward is 132.4 per period versus PaperFCNN 245.4, a -46.0% gap; this compact run does not yet meet the match-FCNN requirement.
- Efficiency criterion: compact training time is 0.046 seconds for MonoKAN versus 0.741 seconds for PaperFCNN, about 16.0x faster in this smoke setting.
- Interpretability criterion: MonoKAN exposes monotone spline components and exact zero violations; the FCNN and penalty models can appear nonincreasing on the sampled grid but do not encode the order-up-to slope by construction.

Important caveat: these numbers are from compact validation artifacts. They verify the pipeline and reveal model behavior, but they are not final paper-replication evidence. The full paper_replication profile is needed for scientific comparison.

Acceptance item	Current compact result
Architecture-enforced monotonicity	Pass
Reward no worse than FCNN by 0.5%	Fail
Primary next action	Tune MonoKAN capacity/training under LS before extensions

Mathematical Formulation

Section 2 - lost-sales dynamics, base-stock structure, and diagnostics

State y_t is on-hand inventory and q_t is the purchase quantity. With no lead time and lost sales, available inventory is a_t = y_t + q_t, sales are min(d_t, a_t), and next inventory is y_{t+1} = max(a_t - d_t, 0). The one-period reward used for evaluation is revenue minus purchase and holding costs.

For a Gamma-demand product, the optimal no-lead-time base-stock policy has q*(y) = max(S - y, 0), where S is the critical-quantile order-up-to level. Therefore the desired structural derivative is dq*/dy = -1 while y < S, and dq*/dy = 0 after the cap at zero order.

Monotonicity means y_2 > y_1 implies q_theta(y_2, z) <= q_theta(y_1, z), where z contains product and demand-history features. A penalty baseline adds loss terms for violations. MonoKAN instead restricts the function class so this inequality is enforced by the architecture.

Metric	Definition	Interpretation
Reward gap vs FCNN	100 * (R_model / R_FCNN - 1)	Performance constraint
Raw violation rate	mean[max(q(y_hi)-q(y_lo), 0) > tol]	Monotonicity before deployment caps
Deployed violation rate	same check after action cap	Operational monotonicity
Slope deviation	mean |Delta q / Delta y + 1| in active regior	Order-up-to shape
Order-up-to error	|(q(0)) - S_oracle| or grid analogue	Structural diagnostic only

Graphically, a good policy curve decreases linearly with slope -1 until it reaches zero order. A monotone curve with the wrong slope can still pass violation tests while failing the order-up-to structural test.

Operational Results

Section 3 - compact reward and service metrics

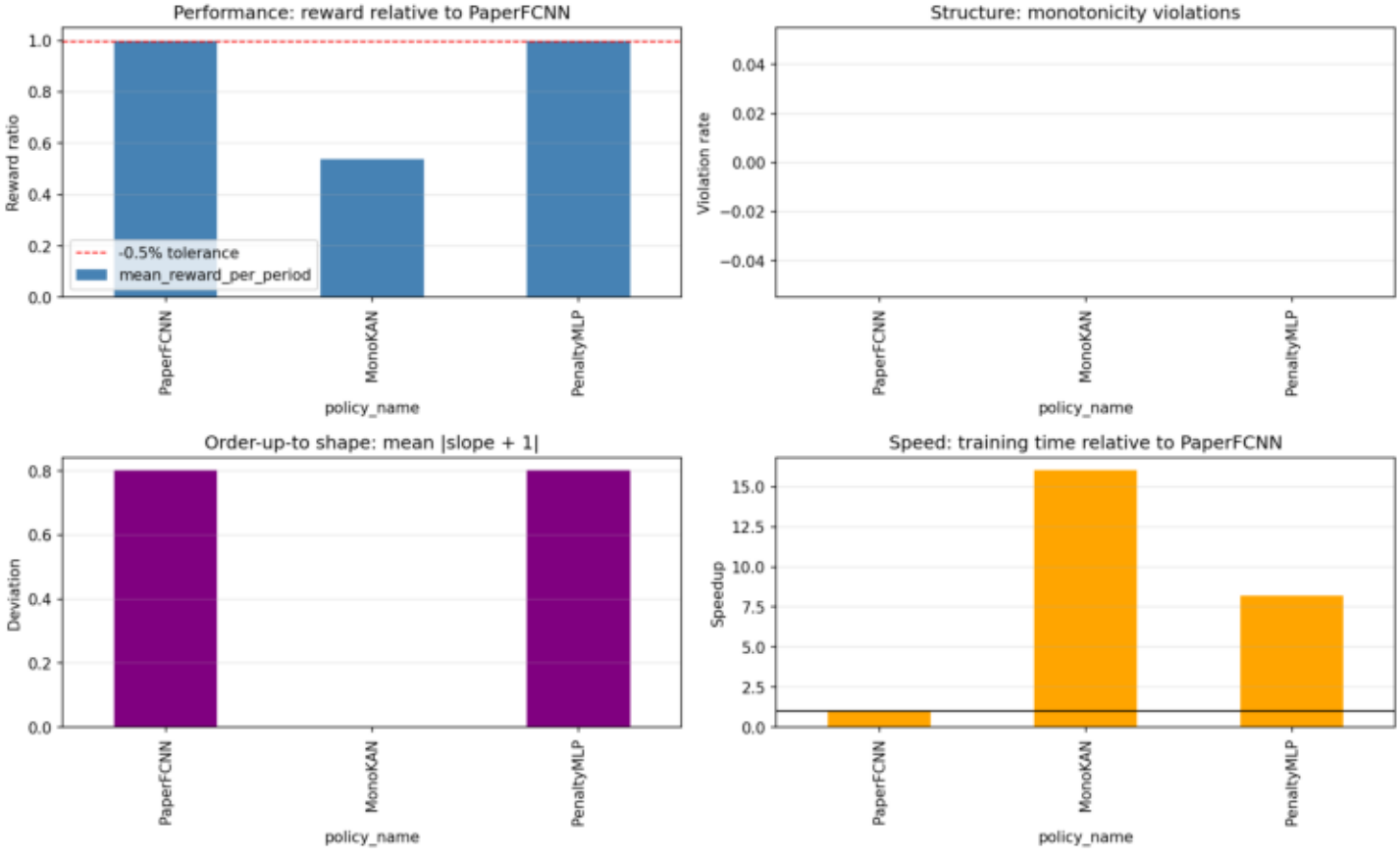
Interpretation: in the compact run, PaperFCNN and PenaltyMLP tie on reward because they share the same initialization and the penalty has no sampled violations. MonoKAN is structurally cleaner but over-orders, creating high inventory and a large reward gap. The compact Table-3-style ranking is not paper-faithful because the product count and horizon are intentionally tiny.

Policy	Reward/period	Service	Lost sales	Mean order	Mean inventory	Train sec
PaperFCNN	245.4	0.951	0.049	12.904	10.109	0.741
MonoKAN	132.4	1.000	0.000	20.500	55.200	0.046
PenaltyMLP	245.4	0.951	0.049	12.904	10.109	0.090
NonOmniscientOptima	197.6	0.601	0.399	10.161	8.612	
OmniscientOptimal	164.1	0.508	0.492	8.444	5.018	
istoricalDemandBaseSt	217.6	0.714	0.286	11.503	13.710	

Policy	Average reward	Gap to omniscient (%)
PaperFCNN	245.4	49.560
MonoKAN	132.4	-19.296
PenaltyMLP	245.4	49.560
NonOmniscientOptimal	197.6	20.423
OmniscientOptimal	164.1	0.000
HistoricalDemandBaseStock	217.6	32.660

Structure Dashboard

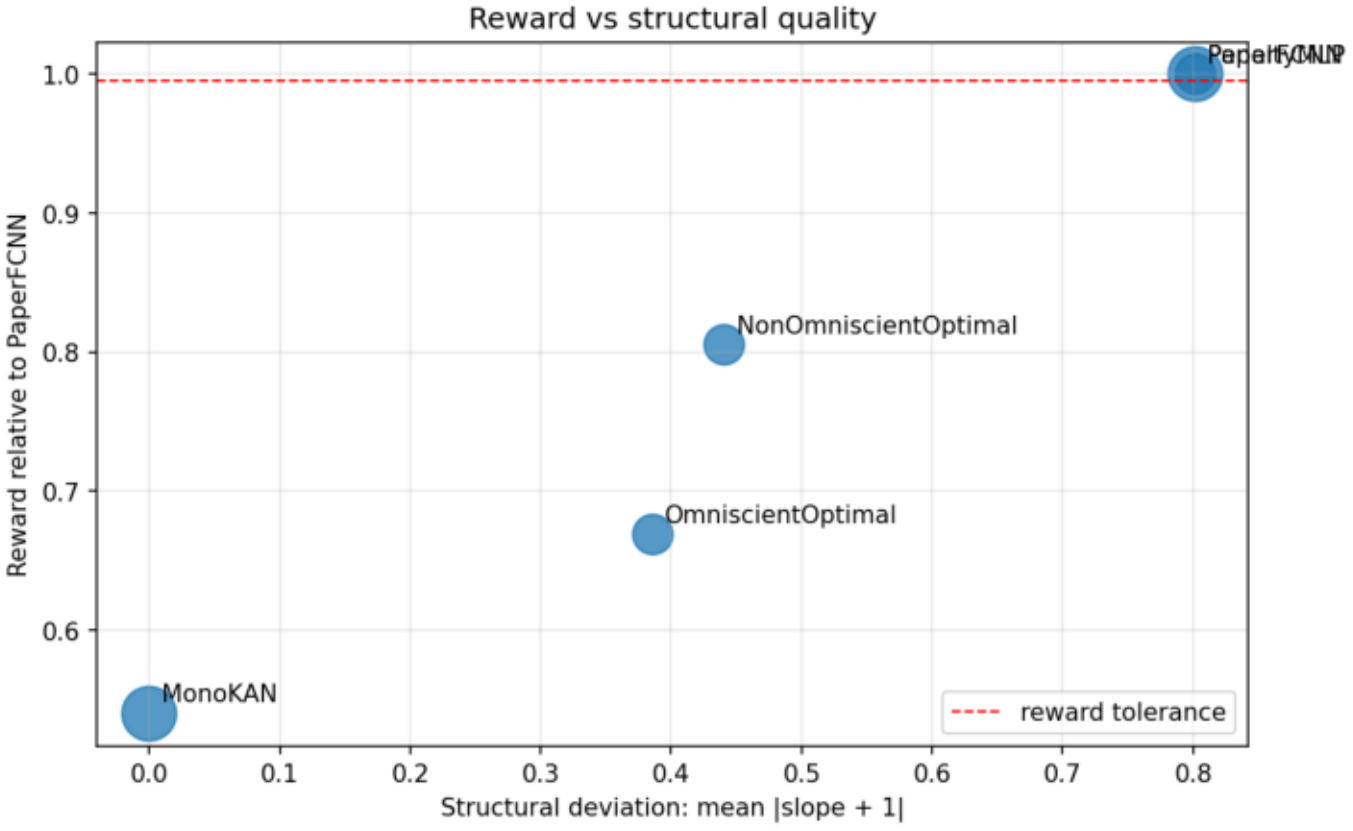
Section 4 - better, faster, interpretable criteria



Mathematical reading: MonoKAN has zero violation rate and zero slope deviation on the diagnostic grid, so it satisfies q(y_hi) <= q(y_lo) exactly in the tested raw and deployed policies.
However, the reward gap is -46.0%, so structure is currently bought at unacceptable reward loss.

Reward-Structure Trade-off

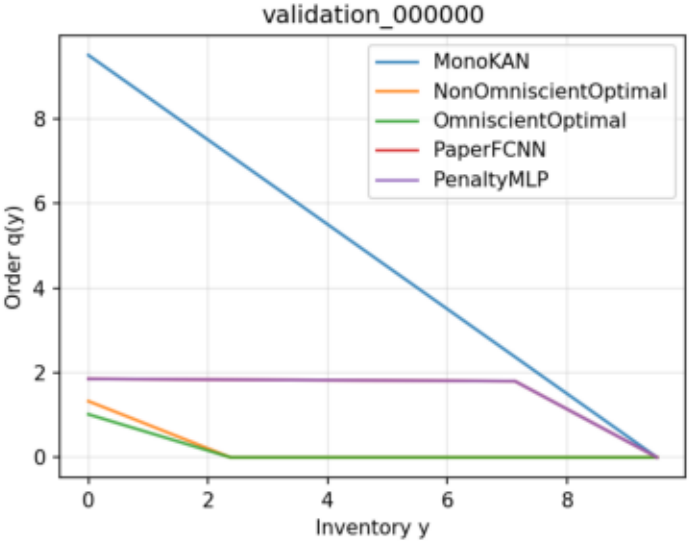
Section 5 - why the project is not just a reward benchmark

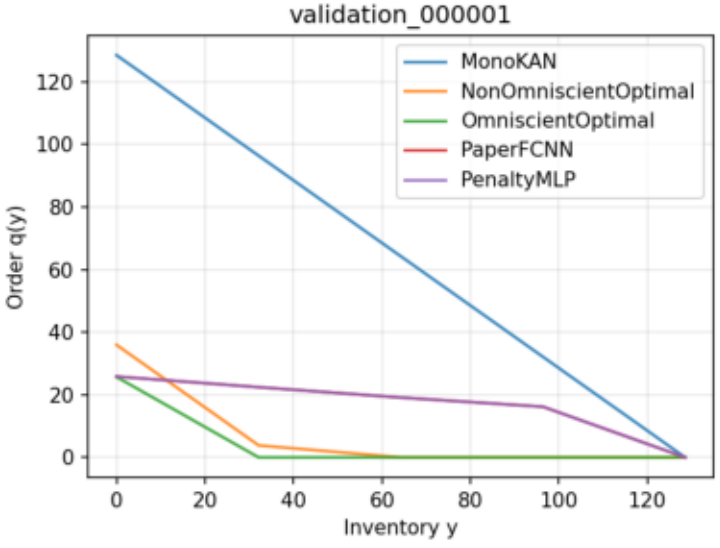


Graphical reading: the desired point is near zero violations and near zero reward gap to PaperFCNN. PenaltyMLP sits on the FCNN performance point in this compact run but does not add architecture-level guarantees. MonoKAN sits at the structural target but not the reward target.

Policy Curves

Section 6 - purchase quantity as a function of inventory

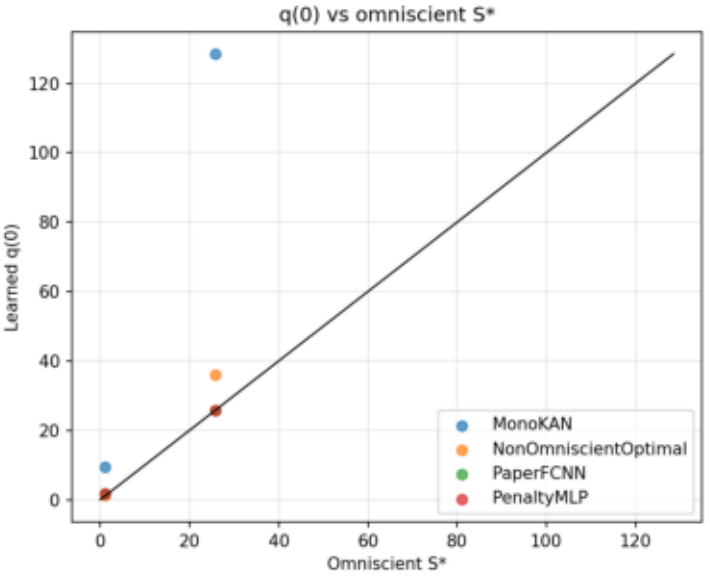


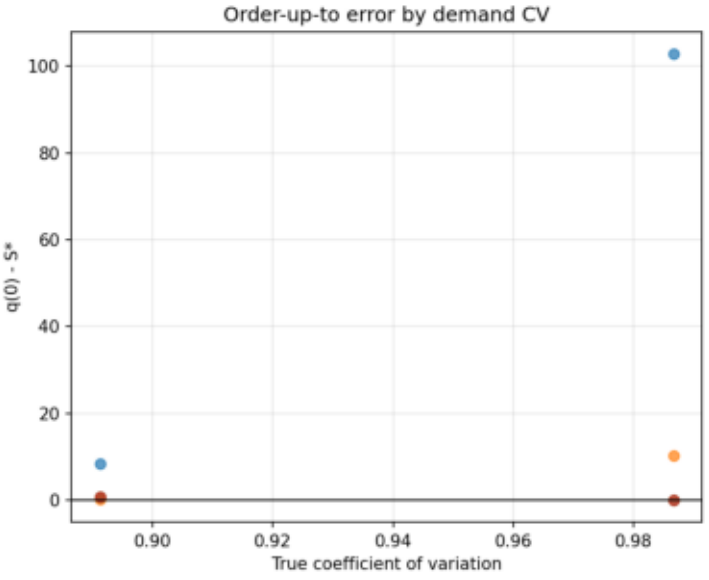


Mathematical reading: for an order-up-to policy, each curve should look like q(y)=max(S-y,0). The important visual signature is a straight line with slope -1 until it hits zero. Smoothness alone is not enough; the curve must have the correct economic slope.

Order-Up-To Diagnostics

Section 7 - structural agreement with Gamma critical quantiles

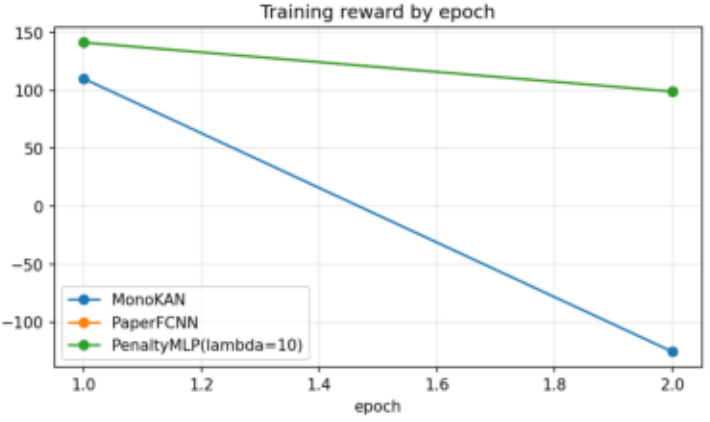


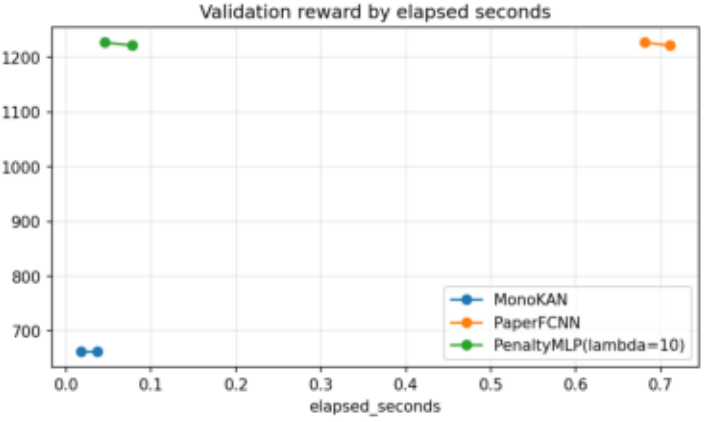


Interpretation: the oracle base-stock level S comes from the exact Gamma critical quantile. Order-up-to error is not used as a supervised training objective; it is an ex-post structural diagnostic showing whether the learned policy recovered the economic form of the solution.

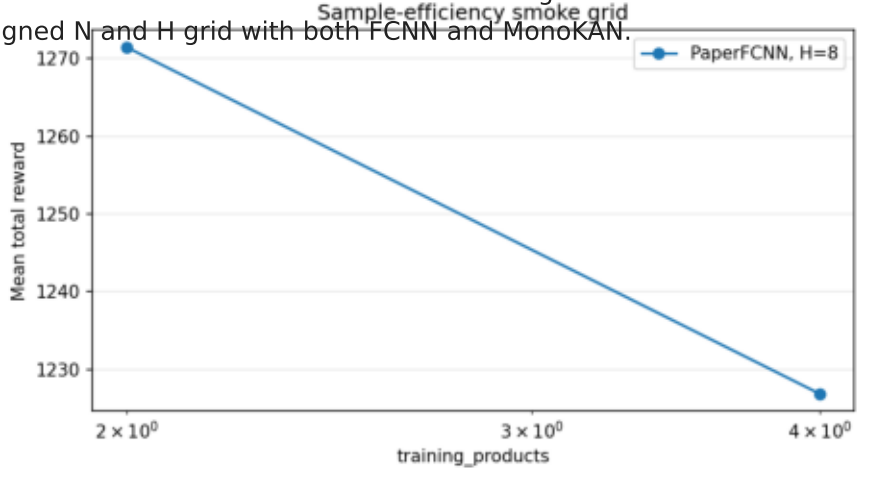
Training And Sample Efficiency

Section 8 - exploratory efficiency evidence





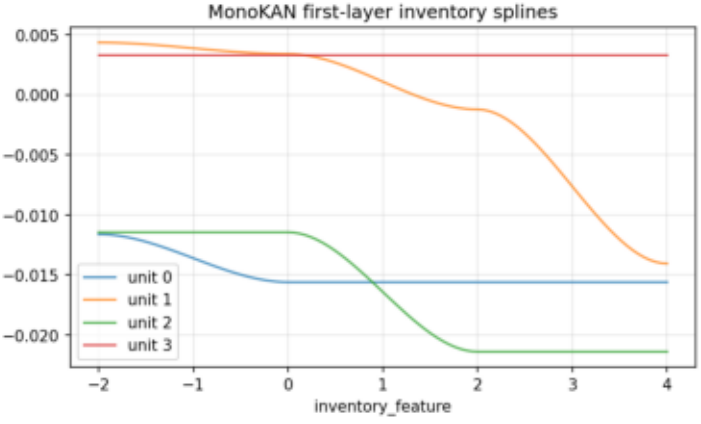
Interpretation: sample efficiency is an open research question. The current compact grid is a smoke test only, so it should not be used to claim faster learning. A credible conclusion requires the paper-aligned N and H grid with both FCNN and MonoKAN.

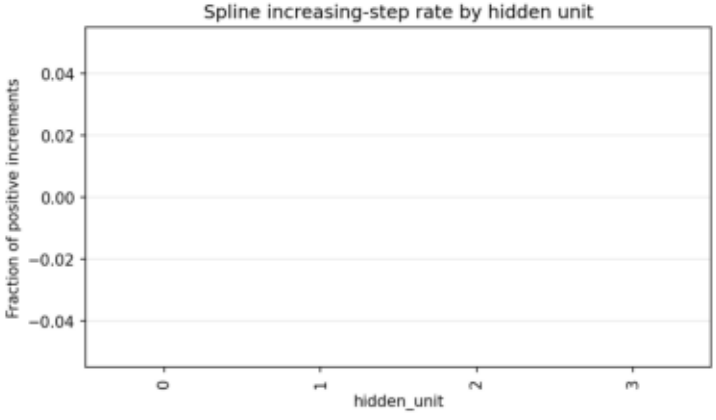


H	N	Policy	Mean total reward	Train sec
8	2	PaperFCNN	1,271.4	0.074
8	4	PaperFCNN	1,226.9	0.108

MonoKAN Spline Interpretability

Section 9 - architecture-level monotone components





Graphical reading: the first-layer inventory splines expose how the inventory coordinate enters the monotone network. This is the main interpretability difference from an unconstrained FCNN: the shape restriction can be inspected directly rather than inferred from penalty terms after training.

Conclusions And Next Steps

Section 10 - project-aligned interpretation

- The current implementation matches the project direction: LS-first, synthesized products, paper-style baselines, architecture-enforced monotonicity, penalty kept as a reference baseline.
- The compact evidence supports the structural claim for MonoKAN: zero raw and deployed violations, plus interpretable splines.
- The compact evidence does not yet support the full project success claim because MonoKAN does not match FCNN reward. This must be fixed before claiming an advantage.
- The penalty baseline is not the proposed method. It remains useful because it mirrors the reference paper's regularization idea and demonstrates why architectural enforcement is a sharper structural intervention.

Step	Action	Reason
1	Run paper_replication LS baseline	Establish credible FCNN and optimal baselines
2	Tune MonoKAN under identical LS data	Close reward gap without adding monotonicity pen
3	Run structure-ls full validation	Produce final policy curves and violation diagnos
4	Run protected final test once	Report held-out result only after model choices are

Bottom line: the report shows mathematically and graphically that architectural monotonicity is working, but the current compact MonoKAN policy is not yet operationally competitive with the FCNN.